

ui.time 全面详解

`ui.time` 是 NiceGUI 中基于 Quasar 的 `QTime` 组件封装的核心时间选择组件，专注于时间点选择功能，支持自定义时间格式、数据绑定、事件回调等基础能力。3.3.0 版本后新增的 `ui.time_input` 是其封装后的一体化输入组件，而 `ui.time` 更适合需要自定义触发方式、布局或深度集成的场景（如弹窗式时间选择、复杂表单内嵌、自定义交互逻辑等）。

一、核心初始化参数

初始化 `ui.time` 仅需 3 个核心参数，简洁覆盖基础时间选择需求，参数设计兼顾灵活性与易用性：

参数名	说明
<code>value</code>	初始时间值，格式需与 <code>mask</code> 匹配（默认 <code>'HH:mm'</code> ，如 <code>'12:00'</code> 、 <code>'08:45'</code> ）
<code>mask</code>	时间字符串格式（默认 <code>'HH:mm'</code> ，支持 Quasar 时间格式占位符，如 <code>'hh:mm A'</code> 12 小时制、 <code>'HH:mm:ss'</code> 带秒数等）
<code>on_change</code>	时间值变更时触发的回调函数，事件对象通过 <code>e.value</code> 获取当前选中时间

基础使用示例

1. 默认格式（24 小时制，HH:mm）

```
from nicegui import ui

result = ui.label('选中时间：')

# 初始化时间选择器，初始值 12:00
ui.time(
    value='12:00',
    mask='HH:mm',
    on_change=lambda e: result.set_text(f'选中时间: {e.value}')
)

ui.run()
```

2. 自定义格式（12 小时制，带上午 / 下午标识）

```
from nicegui import ui

result = ui.label('选中时间: ')

# 12小时制格式（hh:mm A），初始值 03:30 PM
ui.time(
    value='03:30 PM',
    mask='hh:mm A',
    on_change=lambda e: result.set_text(f'选中时间: {e.value}')
)

ui.run()
```

3. 带秒数格式（HH:mm:ss）

```
from nicegui import ui

result = ui.label('选中时间: ')

# 带秒数格式，初始值 14:25:30
ui.time(
    value='14:25:30',
    mask='HH:mm:ss',
    on_change=lambda e: result.set_text(f'选中时间: {e.value}')
)

ui.run()
```

二、组件核心属性

`ui.time` 继承 NiceGUI 基础元素的核心属性，支持动态状态控制、样式自定义、DOM 标识等能力，关键属性如下：

属性名	类型	说明
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观（如宽度、边框、间距）
<code>client</code>	<code>Client</code>	组件所属的客户端实例（多客户端场景下的隔离标识）
<code>enabled</code>	<code>BindableProperty</code>	组件是否启用（可绑定， <code>False</code> 时禁止用户交互，支持动态启用 / 禁用）
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增，用于精准 DOM 操作）
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除（只读属性，用于判断组件生命周期状态）

属性名	类型	说明
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件（只读属性，用于调试或事件控制场景）
<code>parent_slot</code>	<code>Slot None</code>	组件的父插槽（可设置，用于复杂布局中的插槽嵌套）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性（用于扩展时间选择器行为，如 <code>no-arrow-buttons</code> 隐藏增减按钮）
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式（如 <code>width: 200px; margin: 10px 0</code> ）
<code>value</code>	<code>BindableProperty</code>	当前选中的时间值（格式与 <code>mask</code> 一致，支持动态绑定，可通过 <code>set_value</code> 方法修改）
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（支持动态绑定， <code>False</code> 时隐藏组件）

属性操作示例

```
from nicegui import ui

# 初始化时间选择器（12小时制）
time_picker = ui.time(value='09:15 AM', mask='hh:mm A')

# 按钮控制组件状态
ui.button('禁用选择器', on_click=time_picker.disable)
ui.button('启用选择器', on_click=time_picker.enable)
ui.button('设置为 10:30 AM', on_click=lambda: time_picker.set_value('10:30 AM'))
ui.button('隐藏选择器', on_click=lambda: time_picker.set_visibility(False))

ui.run()
```

三、核心特性：自定义触发方式（输入框 + 图标触发）

`ui.time` 本身是纯时间选择面板，无默认输入框，需手动结合 `ui.input`、`ui.menu`、`ui.icon` 实现“输入框 + 图标触发”的交互形式（类似 `ui.time_input`），支持更灵活的布局自定义：

```
from nicegui import ui

# 创建输入框，用于显示时间文本
with ui.input('选择时间') as time_input:
    # 创建菜单，用于包裹时间选择器
    with ui.menu().props('no-parent-event') as menu:
        # 时间选择器与输入框双向绑定（格式：HH:mm）
        with ui.time(mask='HH:mm').bind_value(time_input):
            # 添加关闭按钮（菜单默认无关闭按钮，手动提升体验）
            with ui.row().classes('justify-end mt-2'):
                ui.button('关闭', on_click=menu.close).props('flat')
    # 在输入框右侧添加图标，点击图标打开菜单
    with time_input.add_slot('append'):
```

```
        ui.icon('access_time').on('click', menu.open).classes('cursor-pointer text-gray-500')

    ui.run()
```

关键说明

- `ui.menu().props('no-parent-event')`：防止点击输入框时误触发菜单打开，仅通过图标触发；
- `bind_value(time_input)`：实现时间选择器与输入框的双向数据同步，修改任一组件的值都会同步到另一组件；
- 可通过修改输入框的 `placeholder`、`classes` 等属性，进一步自定义外观。

四、核心方法

`ui.time` 提供完善的方法用于组件状态控制、数据绑定、样式自定义等，按功能分类如下：

1. 状态控制方法

用于直接修改组件的启用状态、可见性、值等基础属性：

方法名	说明
<code>enable()</code>	启用组件（允许用户交互，可选择 / 修改时间）
<code>disable()</code>	禁用组件（禁止用户交互，组件变灰，无法操作）
<code>set_enabled(value: bool)</code>	动态设置启用状态（ <code>True</code> 启用， <code>False</code> 禁用）
<code>set_value(value: str)</code>	设置时间值（需与 <code>mask</code> 格式一致，如 <code>'15:40'</code> 、 <code>'02:15 PM'</code> ）
<code>set_visibility(visible: bool)</code>	动态设置组件可见性（ <code>True</code> 显示， <code>False</code> 隐藏）
<code>clear()</code>	清除所有子元素（极少用于时间选择器，适用于嵌套复杂内容场景）
<code>delete()</code>	删除组件及其所有子元素（释放资源，生命周期结束）

2. 数据绑定方法

支持将组件的 `enabled`、`value`、`visible` 等属性与目标对象属性进行单向 / 双向绑定，实现数据同步：

方法名	说明
<code>bind_enabled(...)</code>	双向绑定组件启用状态与目标对象属性
<code>bind_enabled_from(...)</code>	单向绑定（目标对象 → 组件）启用状态
<code>bind_enabled_to(...)</code>	单向绑定（组件 → 目标对象）启用状态
<code>bind_value(...)</code>	双向绑定组件时间值与目标对象属性（核心绑定方法，支持格式匹配）
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）时间值

方法名	说明
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）时间值
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性

绑定示例（与数据类同步）

```
from dataclasses import dataclass
from nicegui import ui

# 定义数据类（存储业务数据）
@dataclass
class Task:
    remind_time: str = '09:00' # 与时间选择器绑定的属性

task = Task()

# 时间选择器与 task.remind_time 双向绑定（格式：HH:mm）
time_picker = ui.time(mask='HH:mm').bind_value(task, 'remind_time')

# 显示绑定的当前值
ui.label().bind_text_from(task, 'remind_time', lambda v: f'提醒时间: {v}')

# 按钮修改数据类属性（间接同步到时间选择器）
ui.button('推迟30分钟', on_click=lambda: setattr(task, 'remind_time', '09:30'))

ui.run()
```

3. 样式与扩展方法

用于自定义组件外观、添加资源或辅助功能：

方法名	说明
<code>classes(add/remove/toggle/replace)</code>	新增 / 移除 / 切换 / 替换组件的 HTML 类（如 <code>time_picker.classes('w-40 border-blue-500')</code> ）
<code>style(add/remove/replace)</code>	新增 / 移除 / 替换组件的内联 CSS 样式（如 <code>time_picker.style('font-size: 14px;')</code> ）
<code>default_classes(...)</code>	全局修改该类组件的默认 HTML 类（需在实例化前调用，如统一设置宽度）
<code>default_style(...)</code>	全局修改该类组件的默认 CSS 样式（需在实例化前调用）
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示（鼠标悬浮时显示，如 <code>tooltip('选择提醒时间')</code> ）
<code>add_resource(path)</code>	为组件添加资源文件（如自定义 CSS/JS，用于扩展样式或功能）

方法名	说明
<code>mark(*markers)</code>	为组件添加标记（用于测试查询或依赖管理）

样式自定义示例

```
# 全局设置所有时间选择器的默认样式（实例化前调用）
ui.time.default_classes(add='shadow-sm border-gray-300 rounded-lg')
ui.time.default_style(add='margin: 8px 0; padding: 4px;')

# 实例化时添加局部样式
time_picker = ui.time(
    value='16:20',
    mask='HH:mm'
)
time_picker.classes('w-48 border-green-500') # 局部类（宽度、边框颜色）
time_picker.style('font-size: 15px;') # 局部内联样式（字体大小）
time_picker.tooltip('支持 24 小时制，点击选择时间') # 添加提示

ui.run()
```

4. 其他实用方法

方法名	说明
<code>ancestors(include_self)</code>	迭代组件的祖先元素（ <code>include_self=True</code> 包含自身）
<code>descendants(include_self)</code>	迭代组件的子元素（ <code>include_self=True</code> 包含自身）
<code>get_computed_prop(prop_name, timeout)</code>	获取计算属性（需异步等待，如获取组件实际渲染后的宽度）
<code>move(target_container, target_index, target_slot)</code>	移动组件到其他容器（用于动态布局调整）
<code>on(type, handler, ...)</code>	订阅通用 DOM 事件（如点击、鼠标悬浮、聚焦等）
<code>on_value_change(callback)</code>	绑定时间值变更事件（与初始化 <code>on_change</code> 参数等价）
<code>remove(element)</code>	移除子元素（极少使用）
<code>run_method(name, *args)</code>	运行客户端方法（如调用底层 Quasar 组件的原生方法）
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据，如动态修改样式后刷新）

五、事件处理

1. 核心事件：时间值变更（on_change/on_value_change）

时间值变更事件是 `ui.time` 的核心事件，支持两种绑定方式（功能等价）：

```
# 方式1: 初始化时通过 on_change 参数绑定
ui.time(
    value='12:00',
    on_change=lambda e: print(f'时间变更为: {e.value}')
)

# 方式2: 通过 on_value_change 方法绑定
time_picker = ui.time(value='12:00')
time_picker.on_value_change(lambda e: print(f'时间变更为: {e.value}'))
```

2. 通用事件订阅（on 方法）

通过 `on()` 方法订阅其他 DOM 事件（如点击、鼠标悬浮、聚焦等），支持客户端 JS 处理或服务端 Python 处理：

```
from nicegui import ui

time_picker = ui.time(value='09:30', mask='HH:mm')

# 订阅点击事件（服务端处理）
time_picker.on('click', lambda: print('时间选择器被点击'))

# 订阅鼠标悬浮事件（客户端 JS 处理）
time_picker.on(
    'mouseover',
    js_handler='(e) => console.log("鼠标悬浮在时间选择器上")'
)

ui.run()
```

六、版本兼容性说明

特性 / 属性	支持版本
<code>html_id</code> 属性	2.16.0+
<code>toggle</code> 参数（ <code>default_classes</code> ）	2.7.0+
<code>strict</code> 参数（绑定方法）	3.0.0+
同时指定 Python 和 JS 事件处理（ <code>on</code> 方法）	2.18.0+
<code>ui.time_input</code> 替代方案提示	3.3.0+

七、与 ui.time_input 的核心区别

特性	ui.time	ui.time_input
核心形态	纯时间选择面板（无输入框）	输入框 + 时间选择器（一体化组件）
触发方式	需手动结合菜单 / 图标触发	点击输入框自动弹出选择器
适用场景	自定义布局、弹窗式选择、深度配置	表单内嵌、快速实现时间输入功能
初始化复杂度	较高（需手动组合组件）	较低（开箱即用）
核心优势	灵活性强、支持深度定制	便捷性高、集成度高

简单来说：快速实现表单时间输入用 `ui.time_input`；需要自定义触发方式、布局或深度配置时间格式用 `ui.time`。

总结

`ui.time` 是 NiceGUI 中功能基础且灵活的时间选择核心组件，核心优势在于：

- 1. 支持自定义时间格式（24 小时制、12 小时制、带秒数等），适配多样化时间展示需求；
- 2. 可与 `ui.input`、`ui.menu` 等组件自由组合，实现自定义交互逻辑；
- 3. 完善的数据绑定能力，支持跨组件数据同步与业务数据模型对接；
- 4. 样式自定义能力强，可通过类、内联样式适配不同界面风格；
- 5. 基于 Quasar QTime 组件，交互流畅、跨浏览器兼容，格式校验自动生效。

适用于弹窗式时间选择、复杂表单内嵌、自定义时间输入组件等场景，是构建时间交互界面的核心基础组件。